

CSED 312 Project 1 Design Report

김지현(20220302)

전현빈(20220259)

Part 1. Analyze the Current Implementation

We will first examine the main entry point of Pintos before diving into the project. Below is the `pintos_intro` code. Here, we can see the `thread_init`, `thread_start`, and `thread_exit` functions, which will be further explained throughout this design report for Project 1.

```
/** Pintos main entry point. */
int
pintos_init (void)
{
    char **argv;

    /* Clear BSS. */
    bss_init ();

    /* Break command line into arguments and parse options. */
    argv = read_command_line ();
    argv = parse_options (argv);

    /* Initialize ourselves as a thread so we can use locks,
       then enable console locking. */
    thread_init ();
    console_init ();
```

```
/* Greet user. */
printf ("Pintos booting with %"PRIu32" kB RAM...\n",
        init_ram_pages * PGSIZE / 1024);

/* Initialize memory system. */
palloc_init (user_page_limit);
malloc_init ();
paging_init ();

/* Segmentation. */
#ifdef USERPROG
    tss_init ();
    gdt_init ();
#endif

/* Initialize interrupt handlers. */
intr_init ();
timer_init ();
kbd_init ();
input_init ();
#ifdef USERPROG
    exception_init ();
    syscall_init ();
#endif

/* Start thread scheduler and enable interrupts. */
thread_start ();
serial_init_queue ();
timer_calibrate ();

#ifdef FILESYS
```

```

/* Initialize file system. */
ide_init ();
locate_block_devices ();
fileys_init (format_filesys);
#endif

printf ("Boot complete.\n");

if (*argv != NULL) {
    /* Run actions specified on kernel command line. */
    run_actions (argv);
} else {
    // TODO: no command line passed to kernel. Run interactively
}

/* Finish up. */
shutdown ();
thread_exit ();
}

```

1-1. Timer (Alarm Clock)

The current Pintos timer is implemented using a "busy waiting" approach. In this method, the program continuously checks a condition in a loop until the condition is met. While this ensures the timer's accuracy, it leads to unnecessary CPU usage since the program occupies the CPU even when there is no useful work to perform. To optimize the system and reduce CPU waste, our goal is to re-implement the timer in a more efficient way that minimizes CPU usage.

Our task is to implement the `timer_sleep()` function found in `~/devices/timer.c`. To begin, we will first analyze the current implementation of `timer_sleep()`. This function is responsible for putting the calling thread to sleep for a specified number of timer ticks. However, in its current form, it relies on the "busy waiting" method, where the thread continuously checks if the desired sleep duration has passed as seen in the while condition below, leading to unnecessary CPU usage.

```
void
timer_sleep (int64_t ticks)
{
    int64_t start = timer_ticks ();

    ASSERT (intr_get_level () == INTR_ON);
    while (timer_elapsed (start) < ticks)
        thread_yield ();
}
```

Now, we will walk through the implementation of the `timer_interrupt` function, which is located in the `timer.c` file. Each time a timer interrupt occurs, the global variable `ticks` is incremented. This variable keeps track of the total number of clock ticks since the system booted. After incrementing `ticks`, the `thread_tick` function is called to manage various aspects of thread execution.

```
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
```

```
ticks++;      // Increments the global tick count
thread_tick (); // Calls thread_tick to manage thread-related activities
}
```

Now, let's examine other functions used within the `timer_sleep()` function. First, the `timer_ticks` function disables interrupts to ensure that once the OS is booted, the timer operates without being interrupted by other threads or interrupt handlers. It then stores the current value of ticks, which tracks the elapsed time since the system booted. Afterward, interrupts are re-enabled using the `intr_set_level()` function. This function returns the value of ticks, providing a snapshot of the system's current time in ticks.

```
int64_t
timer_ticks (void)
{
    enum intr_level old_level = intr_disable ();
    int64_t t = ticks;
    intr_set_level (old_level);
    return t;
}
```

The `timer_elapsed` function calculates and returns the amount of time that has passed from a specified point ("then") to the current moment (now). In the context of the `timer_sleep` function, `timer_elapsed` is used to determine how much time has passed since the thread began sleeping, from the initial call to the current time.

```
int64_t
timer_elapsed (int64_t then)
{
    return timer_ticks () - then;
}
```

Lastly, the `thread_yield` function allows the CPU to switch to a different thread by changing the state of the current thread from running to ready. First, it ensures that no interrupt handler is currently running. Then, interrupts are disabled to ensure safe scheduling. If the current thread is not idle, its state is updated to ready, and it is added to the list of ready threads. After this, the scheduler is called to shift the CPU's execution to another thread. Finally, interrupts are re-enabled.

An exception is made for the idle thread because, by definition, the idle handler runs only when there are no other threads available to execute.

```
void
thread_yield (void)
{
    struct thread *cur = thread_current ();
    enum intr_level old_level;

    ASSERT (!intr_context ());

    old_level = intr_disable ();
    if (cur != idle_thread)
        list_push_back (&ready_list, &cur->elem);
    cur->status = THREAD_READY;
    schedule ();
}
```

```
    intr_set_level (old_level);  
}
```

The following code defines the states of a thread using an enumeration. First, the running state indicates that the thread is currently executing and only one thread can have `THREAD_RUNNING` state at a time. The ready state represents a thread that is not yet running but is ready to execute as soon as the scheduler assigns it CPU time. (CPU time is a measure of the usage of CPU processing power during the run of a program) The blocked state means the thread is waiting for a specific event to occur before it can proceed. Finally, the dying state signifies that the thread is in the process of terminating.

```
enum thread_status  
{  
    THREAD_RUNNING,    /* Running thread. */  
    THREAD_READY,      /* Not running but ready to run. */  
    THREAD_BLOCKED,    /* Waiting for an event to trigger. */  
    THREAD_DYING       /* About to be destroyed. */  
};
```

1-2. Thread (Alarm Clock)

The following code specifies the information needed for each thread. The `tid` stores the unique identification number of each thread. The `status` holds the thread's current status, based on the enumeration explained above.

When multiple threads run simultaneously, each is in a different stage of execution. Therefore, each thread needs its own stack to store local variables, function calls,

and so on. In other words, every thread has its own stack, and the stack pointer points to this stack memory for each thread. When a thread is in the running state, the CPU's stack pointer register points to the top of the stack, and this member variable is not used. However, when the CPU switches to a different thread, this member variable stores the stack pointer of the thread.

The priority, which will be explained in the "Priority Scheduling" section, determines the order in which ready threads are executed next.

The allelem list contains all threads, while the elem list holds threads in the same state. For example, there could be a list for blocked threads and a separate list for ready threads. The pagedir is used for mapping virtual memory space to physical memory space, but this will not be relevant in Project 1.

Lastly, the magic field is used to detect stack overflow, so it is placed as the last element of the struct thread. In the thread_current function, the magic variable is checked against THREAD_MAGIC. If they do not match, a stack overflow has occurred.

In this section, our focus is on the status field, where the current state of the thread is saved, and the thread lists that manage the threads in various states.

```
struct thread
{
    tid_t tid;                /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16];            /* Name (for debugging purposes). */
    uint8_t *stack;           /* Saved stack pointer. */
    int priority;              /* Priority. */
    struct list_elem allelem;  /* List element for all threads list. */
}
```



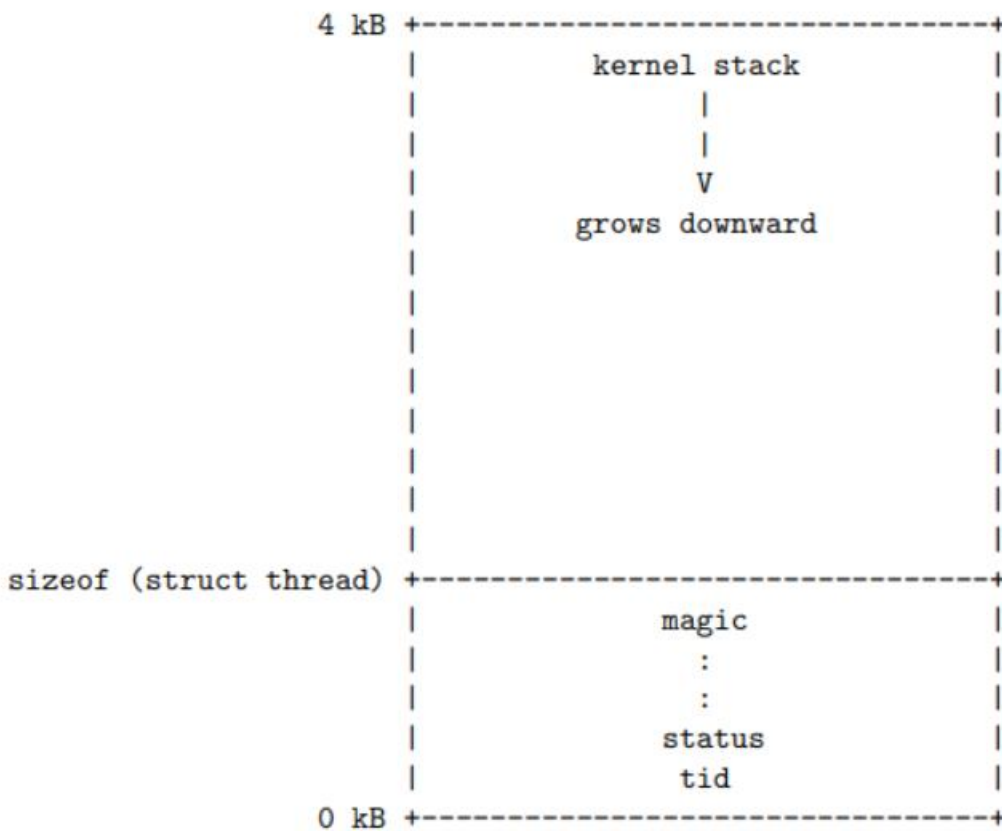
```

struct list_elem elem;          /* List element. */

#ifdef USERPROG
    uint32_t *pagedir;          /* Page directory. */
#endif

    unsigned magic;              /* Detects stack overflow. */
};

```



Now, let's examine how the thread system is started using the `thread_start` function. First, a semaphore object, `idle_started`, is initialized to 0, meaning the semaphore is locked. This object is used to check whether the idle thread has been initialized. The

concept of a semaphore will be explained further in the "2. Priority Scheduling" section.

Next, a thread named Idle is created using the `thread_create` function, which is explained in detail below. The idle thread runs when no other threads are in the ready list. Afterward, interrupts are enabled, and it waits until the initialization of the idle thread is complete.

```
void
thread_start (void)
{
    /* Create the idle thread. */
    struct semaphore idle_started;
    sema_init (&idle_started, 0);
    thread_create ("idle", PRI_MIN, idle, &idle_started);

    /* Start preemptive thread scheduling. */
    intr_enable ();

    /* Wait for the idle thread to initialize idle_thread. */
    sema_down (&idle_started);
}
```

Next, we will examine how the thread data structure is initialized in the `init_thread` function. First, the current interrupt level is saved in the `old_level` variable. Then, all the values in thread `t` are set to zero using the `memset(dst, value, size)` function, which "sets the specified number of bytes (SIZE) in the destination (DST) to the given value (VALUE)". After that, the current state of the thread is set

to BLOCKED (or waiting as explained in the lecture). The thread's name, stack pointer, priority, and magic number are then initialized. Finally, interrupts are disabled, the current thread is added to the allelem list, and interrupts are re-enabled.

```
static void
init_thread (struct thread *t, const char *name, int priority)
{
    enum intr_level old_level;

    ASSERT (t != NULL);
    ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
    ASSERT (name != NULL);

    memset (t, 0, sizeof *t);
    t->status = THREAD_BLOCKED;
    strcpy (t->name, name, sizeof t->name);
    t->stack = (uint8_t *) t + PGSIZE;
    t->priority = priority;
    t->magic = THREAD_MAGIC;

    old_level = intr_disable ();
    list_push_back (&all_list, &t->allelem);
    intr_set_level (old_level);
}
```

The following function initializes the overall thread system. In other words, this function specifically handles the very first thread that runs when the program is

executed. First, it initializes `tid_lock`, `ready_list`, and `all_list`. Initializing the lock prevents multiple threads from sharing the same identification number. As we will see in the `thread_create` function later, when a thread is first created, it will be in the `BLOCKED` state, so the `ready_list` must be empty initially. Additionally, there shouldn't be any threads in the `all_list` at this point.

Next, the currently running thread, named "main", is initialized using the previously mentioned `init_thread` function. The status of this thread is then set to `THREAD_RUNNING`, and a unique thread identification number (`tid`) is allocated.

```
void
thread_init (void)
{
    ASSERT (intr_get_level () == INTR_OFF);

    lock_init (&tid_lock);
    list_init (&ready_list);
    list_init (&all_list);

    /* Set up a thread structure for the running thread. */
    initial_thread = running_thread ();
    init_thread (initial_thread, "main", PRI_DEFAULT);
    initial_thread->status = THREAD_RUNNING;
    initial_thread->tid = allocate_tid ();
}
```

The `thread_tick` function, which is called within `timer_interrupt`, is responsible for maintaining statistics on thread execution and ensuring that thread preemption

occurs when necessary. There are three types of threads: idle, user, and kernel threads. First, the function retrieves the current thread and checks whether it is idle, user, or kernel. Then, it increments `idle_ticks`, `user_ticks`, or `kernel_ticks` accordingly. Next, in the if statement, the function checks whether `thread_ticks` — which tracks how much time has been spent on the current thread — has exceeded `TIME_SLICE`, the maximum amount of time a thread can run. If so, it calls the `intr_yield_on_return` function to stop the execution of the current thread and yield control to another thread.

```
void
thread_tick (void)
{
    struct thread *t = thread_current(); // Get the currently running thread

    /* Update statistics. */
    if (t == idle_thread)
        idle_ticks++; // If the current thread is idle, increment idle_ticks
#ifdef USERPROG
    else if (t->pagedir != NULL)
        user_ticks++; // If the current thread is a user thread, increment user_ticks
#endif
    else
        kernel_ticks++; // Otherwise, it's a kernel thread, so increment kernel_ticks

    /* Enforce preemption. */
    if (++thread_ticks >= TIME_SLICE)
        intr_yield_on_return(); // If the thread has used up its time slice, preempt it
}
```

Finally, let's look at the `thread_create` function, which is called in the `thread_start` function. First, memory is allocated for the thread and initialized to 0 using the `palloc_get_page` function. The thread is then initialized using the previously explained `init_thread` function. The `allocate_tid` function assigns a unique identification number to the current thread.

Next, the function sets up the `kernel_thread_frame`, the `switch_entry_frame`, and the `switch_thread_frame`.

When setting up the `kernel_thread_frame`, the thread needs to know what function to execute and what data (stored in `aux`) is required. Here, `eip` of `kernel_thread_frame` is the instruction pointer that determines where the code starts, and it will be set by the `kernel_thread` function.

The `eip` of `switch_entry_frame` is set to the `kernel_thread` function. This part of the code instructs the CPU on what to do when the thread is first executed in kernel mode.

Lastly, the `switch_thread_stack` frame is prepared for context switching. The `switch_entry` frame contains the function that will be executed when a context switch occurs. Since this is the first time the thread is created, the `ebp` will be set to 0 (normally, the `ebp` points to the previous stack frame when the thread is called from another function).

Finally, the current thread is added to the ready list using the `thread_unblock` function, and the thread id is returned.

<code>tid_t</code> <code>thread_create (const char *name, int priority,</code>

```

        thread_func *function, void *aux)
{
    struct thread *t;
    struct kernel_thread_frame *kf;
    struct switch_entry_frame *ef;
    struct switch_threads_frame *sf;
    tid_t tid;

    ASSERT (function != NULL);

    /* Allocate thread. */
    t = palloc_get_page (PAL_ZERO);
    if (t == NULL)
        return TID_ERROR;

    /* Initialize thread. */
    init_thread (t, name, priority);
    tid = t->tid = allocate_tid ();

    /* Stack frame for kernel_thread(). */
    kf = alloc_frame (t, sizeof *kf);
    kf->eip = NULL;
    kf->function = function;
    kf->aux = aux;

    /* Stack frame for switch_entry(). */
    ef = alloc_frame (t, sizeof *ef);
    ef->eip = (void (*) (void)) kernel_thread;

    /* Stack frame for switch_threads(). */
    sf = alloc_frame (t, sizeof *sf);

```

```
sf->eip = switch_entry;
sf->ebp = 0;

/* Add to run queue. */
thread_unblock (t);

return tid;
}
```

Since we have explored how to create a thread, we now need to understand what happens when a thread exits. Then, interrupts are disabled to ensure that the thread exit process is not interrupted by any other events that could interfere with the process. Disabling interrupts guarantees atomicity, meaning that the entire thread exit process can complete without interruption, ensuring consistency and preventing race conditions.

Next, the thread is removed from the allelem list (which contains all threads). The state of the current thread is then set to `THREAD_DYING`, and the CPU is rescheduled to switch to a different thread.

Finally, the function uses `NOT_REACHED` to ensure that the code is functioning correctly, as this point in the code should never be reached.

```
void
thread_exit (void)
{
    ASSERT (!intr_context ());

#ifdef USERPROG
```

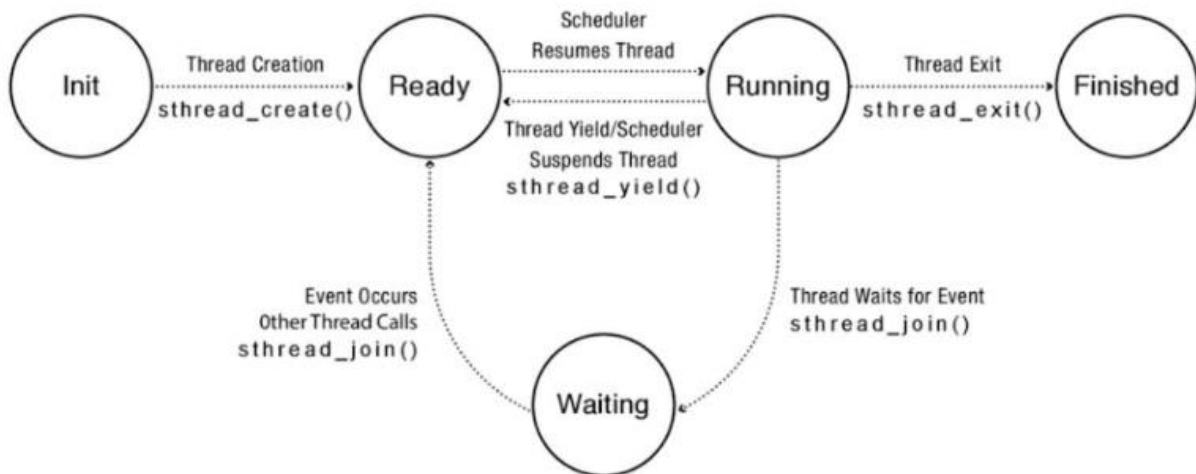


```

    process_exit ();
#endif

/* Remove thread from all threads list, set our status to dying,
   and schedule another process. That process will destroy us
   when it calls thread_schedule_tail(). */
intr_disable ();
list_remove (&thread_current()->allelem);
thread_current ()->status = THREAD_DYING;
schedule ();
NOT_REACHED ();
}

```



Finally, the hint suggests using the `thread_block` and `thread_unblock` functions, so let's take a closer look at them. The `thread_block` function changes the state of the current thread to "blocked" and re-schedules the CPU priorities for execution. On the other hand, the `thread_unblock` function changes the state of a thread to "ready" and adds it to the ready list.

```

void
thread_block (void)
{
    ASSERT (!intr_context ());
    ASSERT (intr_get_level () == INTR_OFF);

    thread_current ()->status = THREAD_BLOCKED;
    schedule ();
}

void
thread_unblock (struct thread *t)
{
    enum intr_level old_level;

    ASSERT (is_thread (t));

    old_level = intr_disable ();
    ASSERT (t->status == THREAD_BLOCKED);
    list_push_back (&ready_list, &t->elem);
    t->status = THREAD_READY;
    intr_set_level (old_level);
}

```

2-1. Thread Switching (Priority Scheduling)

Our objective is to replace the existing round-robin scheduler with a priority-based scheduler. To achieve this, we must first examine the functions involved in the scheduling process. Since we intend to utilize the `thread_block` and `thread_unblock` functions, we should identify the code where a

thread is added to the ready list. In particular, the `thread_unblock` function adds the unblocked thread to the ready list using the `list_push_back` function. Therefore, our initial step will be to analyze how the `list_push_back` function operates.

The `list_push_back` function is located in `/lib/kernel/list.c`. This function identifies the end of the list and inserts a new element at that position using the `list_insert` function.

```
void
list_push_back (struct list *list, struct list_elem *elem)
{
    list_insert (list_end (list), elem);
}

struct list_elem *
list_end (struct list *list)
{
    ASSERT (list != NULL);
    return &list->tail;
}

void
list_insert (struct list_elem *before, struct list_elem *elem)
{
    ASSERT (is_interior (before) || is_tail (before));
    ASSERT (elem != NULL);

    elem->prev = before->prev;
    elem->next = before;
```

```
before->prev->next = elem;
before->prev = elem;
}
```

As seen in the Alarm Clock section, both `thread_yield` and `thread_block` utilize the `schedule` function to transfer control of the CPU from the current thread to the next thread. In this function, `cur` points to the current running thread, while `next` points to the next thread in the ready list. Using the `switch_threads` function, the CPU is switched according to thread priority.

There are three thread functions that require a switch: `thread_block`, `thread_exit`, and `thread_yield`. Before these functions call `schedule`, interrupts must be disabled, and the state of the running thread must be changed to another state.

```
static void
schedule (void)
{
    struct thread *cur = running_thread ();
    struct thread *next = next_thread_to_run ();
    struct thread *prev = NULL;

    ASSERT (intr_get_level () == INTR_OFF);
    ASSERT (cur->status != THREAD_RUNNING);
    ASSERT (is_thread (next));

    if (cur != next)
        prev = switch_threads (cur, next);
    thread_schedule_tail (prev);
}
```

The next question is: how is the next thread determined, and how is the switch made? The next thread is selected by the `next_thread_to_run` function, which returns the first element in the ready list using the `list_pop_front` function.

```
static struct thread *
next_thread_to_run (void)
{
    if (list_empty (&ready_list))
        return idle_thread;
    else
        return list_entry (list_pop_front (&ready_list), struct thread, elem);
}
```

The `switch_threads` function is an assembly routine located in `thread/switch.S`. It saves the registers to the stack and stores the current CPU stack pointer in the `stack` member of the current struct thread. Then, it loads the stack pointer from the `stack` member of the new thread and restores the register values from the stack.

In the `switch_threads` function, the `CUR` thread, which is the currently running thread, switches its context to the `NEXT` thread. During the switch, the function returns `CUR` in `NEXT`'s context.

```
struct thread *switch_threads (struct thread *cur, struct thread *next);
```

The `thread_schedule_tail` function is responsible for preparing the new thread for execution and releasing resources used by the previous thread if it has exited.

First, a pointer `cur` is set to point to the currently running thread, and the state of the current thread is updated to `THREAD_RUNNING`. The scheduler timer is reset by setting `thread_ticks` to 0. The `USERPROG` section applies when there is a user program, but we will not focus on this part for now.

Next, the function frees the memory pages used by any thread that has exited. Using an if statement, it checks whether the previous thread exists, whether the previous thread is in the `THREAD_DYING` state, and whether it is not the initial thread. The initial thread never needs to be freed during program execution. If these conditions are met, the function further checks that the previous thread is not the current thread, and then releases the memory used by the previous thread.

```
void
thread_schedule_tail (struct thread *prev)
{
    struct thread *cur = running_thread ();

    ASSERT (intr_get_level () == INTR_OFF);

    /* Mark us as running. */
    cur->status = THREAD_RUNNING;

    /* Start new time slice. */
    thread_ticks = 0;

#ifdef USERPROG
    /* Activate the new address space. */
    process_activate ();
#endif
}
```

```

/* If the thread we switched from is dying, destroy its struct
   thread. This must happen late so that thread_exit() doesn't
   pull out the rug under itself. (We don't free
   initial_thread because its memory was not obtained via
   palloc().) */
if (prev != NULL && prev->status == THREAD_DYING && prev != initial_thread)
{
    ASSERT (prev != cur);
    palloc_free_page (prev);
}
}

```

When a new thread is created in `thread_create`, it has never run before, so it doesn't have a way to be executed by `switch_threads`. This is why `thread_create` places three fake stack frames on the new thread's stack - `kernel_thread_frame`, `switch_entry_frame`, and `switch_threads_frame`.

We need to modify the above scheduling process so that the threads in the ready list are lined up according to their priority. This ensures that higher-priority threads are selected for execution before lower-priority ones.

2-2. Synchronization (Priority Scheduling)

Additionally, there are priority-sema and priority-condvar tests, which focus on how semaphores work. A semaphore ensures that a process waits until it receives a specific signal. Since this section addresses synchronization, we can refer to the `/threads/synch.h` file. This file declares three key structures: semaphore, lock, and condition.

First, a semaphore controls an access to the shared resources among the multiple processes or threads. It signals when the resources are available or not. Below, the semaphore structure holds an unsigned integer called "value" and a list of waiting threads.

```
struct semaphore
{
    unsigned value;          /**< Current value. */
    struct list waiters;     /**< List of waiting threads. */
};
```

The `sema_init` function first asserts that the `sema` pointer is not null, ensuring that the semaphore has been properly allocated. It then sets the semaphore's initial value to the provided value and initializes the waiters list, which will be used to keep track of threads waiting on the semaphore.

```
void
sema_init (struct semaphore *sema, unsigned value)
{
    ASSERT (sema != NULL);

    sema->value = value;
    list_init (&sema->waiters);
}
```


A semaphore has two key functions: `sema_down` and `sema_up`.

The `sema_down` function waits for the semaphore's value to become positive and otherwise, decrements the value by 1. Inside the function, within the while loop, the current thread's element is added to the back of the semaphore's waiting list using the `list_push_back` function (as explained above). After that, the thread is blocked while it waits for the semaphore to become available.

The `sema_up` function, on the other hand, retrieves the first element from the semaphore's waiting list and unblocks the corresponding thread, allowing it to proceed. Then, increase the value by 1.

For both `sema_down` and `sema_up`, we first check whether the semaphore argument is not null, ensuring that the function is not called during interrupt handling. Next, interrupts are disabled to maintain atomicity during the execution of both functions. Once all tasks are completed, interrupts are re-enabled.

```
void
sema_down (struct semaphore *sema)
{
    enum intr_level old_level;

    ASSERT (sema != NULL);
    ASSERT (!intr_context ());

    old_level = intr_disable ();
    while (sema->value == 0)
    {
        list_push_back (&sema->waiters, &thread_current ()->elem);
        thread_block ();
    }
}
```

```

    }
    sema->value--;
    intr_set_level (old_level);
}

void
sema_up (struct semaphore *sema)
{
    enum intr_level old_level;

    ASSERT (sema != NULL);

    old_level = intr_disable ();
    if (!list_empty (&sema->waiters))
        thread_unblock (list_entry (list_pop_front (&sema->waiters),
                                           struct thread, elem));

    sema->value++;
    intr_set_level (old_level);
}

```

Next is the lock. Lock is another synchronization mechanism where only one process or thread can access the critical section at once. This is to prevent a race condition where a shared data is modified concurrently. According to the textbook, a lock "implies mutual exclusion — only one thread at a time can hold the lock." Inside the structure, there is a thread variable called holder, which stores information about the thread that currently holds the lock, and a semaphore variable that controls the lock mechanism.

```
struct lock
{
    struct thread *holder;    /**< Thread holding lock (for debugging). */
    struct semaphore semaphore; /**< Binary semaphore controlling access. */
};
```

The `lock_init` function begins by asserting that the lock pointer is not null, ensuring that memory has been properly allocated for the lock. It then sets the holder of the lock to `NULL`, indicating that no thread currently holds the lock. Finally, the function calls `sema_init()` to initialize the lock's internal semaphore with a value of 1. This semaphore will be used to manage access to the lock, ensuring only one thread can hold the lock at a time.

```
void
lock_init (struct lock *lock)
{
    ASSERT (lock != NULL);

    lock->holder = NULL;
    sema_init (&lock->semaphore, 1);
}
```

As shown below, the two key functions for a lock utilize the `sema_down` and `sema_up` functions. In the `lock_acquire` function, `sema_down` is used to put the current thread in a waiting state if the semaphore's value is 0. If the value is greater than 0, the current thread will acquire the lock and be assigned to the holder variable.

In the `lock_release` function, `sema_up` is used to allow other threads in the lock's waiting list to acquire the lock by setting the holder to null, effectively releasing the lock.

```
void
lock_acquire (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (!lock_held_by_current_thread (lock));

    sema_down (&lock->semaphore);
    lock->holder = thread_current ();
}

void
lock_release (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (lock_held_by_current_thread (lock));

    lock->holder = NULL;
    sema_up (&lock->semaphore);
}
```

The last structure is the condition. Condition variable is a synchronization primitives that keeps the thread waiting until the condition is met. This primitive allows management of access to shared resources along with the lock. This structure

maintains a list of threads that are waiting for certain condition variables to be satisfied.

```
struct condition
{
    struct list waiters;    /**< List of waiting threads. */
};
```

The `cond_init` function first asserts that the `cond` pointer is not null, ensuring that the condition variable has been properly allocated. It then initializes the waiters list, which will be used to keep track of threads waiting on the condition variable.

```
void
cond_init (struct condition *cond)
{
    ASSERT (cond != NULL);

    list_init (&cond->waiters);
}
```

Below are the three key functions for the condition structure. Threads will remain in a waiting state until a specific condition is satisfied.

The `cond_wait` function first initializes the semaphore's value to 0, ensuring that the thread will wait until the `cond_signal` function sends a signal. The thread is then added to the waiting list of the condition structure. After that, the lock held by the thread is released, allowing other threads to access shared resources. Finally, the

thread's state is set to waiting by calling `sema_down`, and it reacquires the lock so that once it wakes up, the resources are secure and ready for use.

The `cond_signal` function removes the first thread from the waiting list in the condition structure and calls `sema_up` to unblock that thread by incrementing the semaphore's value.

The `cond_broadcast` function wakes up all threads waiting on a condition variable. The function first asserts that both `cond` and `lock` are not null, ensuring that valid condition and lock pointers are provided. It then enters a loop that continuously checks if the waiters list is empty. As long as the list is not empty, it calls `cond_signal` to wake up one waiting thread at a time, ensuring that all threads waiting on the condition variable are eventually signaled and allowed to proceed.

```
void
cond_wait (struct condition *cond, struct lock *lock)
{
    struct semaphore_elem waiter;

    ASSERT (cond != NULL);
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (lock_held_by_current_thread (lock));

    sema_init (&waiter.semaphore, 0);
    list_push_back (&cond->waiters, &waiter.elem);
    lock_release (lock);
    sema_down (&waiter.semaphore);
    lock_acquire (lock);
}
```

```
}
```

```
void
```

```
cond_signal (struct condition *cond, struct lock *lock UNUSED)
```

```
{
```

```
    ASSERT (cond != NULL);
```

```
    ASSERT (lock != NULL);
```

```
    ASSERT (!intr_context ());
```

```
    ASSERT (lock_held_by_current_thread (lock));
```

```
    if (!list_empty (&cond->waiters))
```

```
        sema_up (&list_entry (list_pop_front (&cond->waiters),  
                                struct semaphore_elem, elem)->semaphore);
```

```
}
```

```
void
```

```
cond_broadcast (struct condition *cond, struct lock *lock)
```

```
{
```

```
    ASSERT (cond != NULL);
```

```
    ASSERT (lock != NULL);
```

```
    while (!list_empty (&cond->waiters))
```

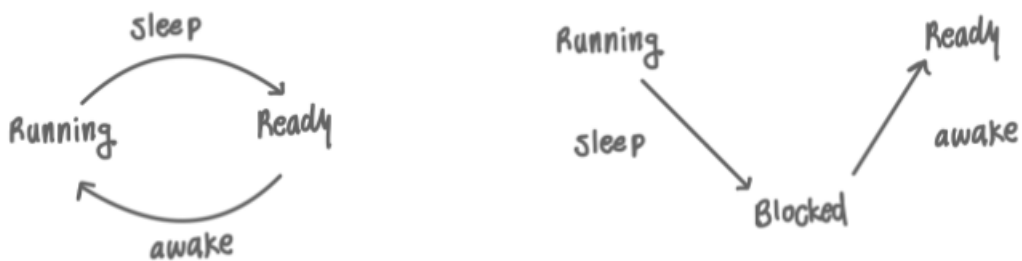
```
        cond_signal (cond, lock);
```

```
}
```

Part 2. Propose Design

1. Alarm Clock

The current busy-waiting design alternates between two states: running and ready, as seen on the leftside of the below picture. However, as suggested by the hint about using block and unblock functions, we will introduce an additional blocked state, as seen on the rightside of the picture.



This will allow the program to avoid checking the condition on every clock tick, thereby improving efficiency. To achieve this, we will create two new functions: `thread_sleep` and `thread_awake`. The `thread_sleep` function will block the thread when it is waiting, while the `thread_awake` function will unblock the thread once the specified number of ticks has elapsed.

In the `timer_sleep` function, we will replace the existing while loop with a call to `thread_sleep`. Additionally, we will modify the `timer_interrupt` function to call `thread_awake` every time a tick elapses, ensuring that blocked threads are woken up at the appropriate time.

2. Priority Scheduling

Priority scheduling goals:

1. The thread with the highest priority must always be scheduled first.

2. If a thread with a higher priority than the currently running thread becomes available, the current thread should immediately yield the CPU.
3. When multiple threads are waiting for synchronization, the thread with the highest priority should be awakened first.
4. If a thread lowers its priority, it should immediately yield the CPU to allow higher-priority threads to run.

As discussed in the analysis of the current implementation, we need to incorporate a priority check for the ready list. There are two potential approaches to achieve this.

First, we can check the priority when adding a new thread to the ready list. Second, we can check the priority when popping an element from the ready list. From a time complexity perspective, the first approach is more efficient because the ready list will always be sorted by priority. In contrast, the second approach requires checking the priority of all threads in the ready list every time an element is popped, which introduces unnecessary overhead.

The `list_push_back` function is used in `thread_unblock`, `thread_yield`, and `init_thread` (`list_push_back` function is introduced in the analysis part). However, since `init_thread` is called within `thread_create`, where `thread_unblock` is also used, we only need to focus on modifying `thread_unblock` and `thread_yield`. In the code file where `list_push_back` is located (`/lib/kernel/list.c`), there is a `list_insert_ordered` function that we can utilize to implement the first approach mentioned earlier, ensuring that threads are inserted into the ready list based on their priority.

```

void
list_insert_ordered (struct list *list, struct list_elem *elem,
                    list_less_func *less, void *aux)
{
    struct list_elem *e;

    ASSERT (list != NULL);
    ASSERT (elem != NULL);
    ASSERT (less != NULL);

    for (e = list_begin (list); e != list_end (list); e = list_next (e))
        if (less (elem, e, aux))
            break;
    return list_insert (e, elem);
}

```

At this point, ideally, the threads in the ready list are correctly ordered by priority (goal #1). However, we still need to compare the priority of the currently running thread with the first thread in the ready list, which has the highest priority. To achieve this, we can introduce a new function that compares the priority of these two threads. If the priority of the thread in the ready list is higher than that of the currently running thread, the `thread_yield` function should be called. This check should be added whenever a new thread is forked (i.e., in `thread_create`) and when a thread's priority changes (i.e., in `thread_set_priority`) (goal #2).

According to the pintos documentation, there is a "priority inversion" problem when replacing the current round-robin scheduler with a new priority scheduler, and we need to implement priority donation. Priority inversion occurs when a higher-priority thread is blocked due to the execution of a lower-priority thread. In other words, if a

low-priority thread holds a lock, higher-priority threads will not get CPU time because the CPU scheduler continually searches for the highest-priority thread, but the lock prevents other threads from running.

To resolve this issue, the highest-priority thread should donate its priority to the lowest-priority thread holding the lock. Once the lower-priority thread releases the lock, the highest-priority thread will regain its original priority. We will use the following two functions to implement this. The `set_priority` function sets the current thread's priority to `new_priority`, and the `get_priority` function returns the current thread's priority.

```
/** Sets the current thread's priority to NEW_PRIORITY. */
void
thread_set_priority (int new_priority)
{
    thread_current ()->priority = new_priority;
}

/** Returns the current thread's priority. */
int
thread_get_priority (void)
{
    return thread_current ()->priority;
}
```

Next, we need to address the case involving semaphores and priority donation. To correctly implement the semaphore and condition variable test cases, we replace `list_push_back` with `list_insert_ordered` to maintain proper priority order(goal

#3). Additionally, we incorporate the new function mentioned above (for priority donation) to ensure proper priority comparison and thread yielding when necessary.

To implement a priority donation mechanism that allows multiple donations to occur, a donation list will be introduced, which tracks all threads that have donated their priority to a lower-priority thread.

To do so, we will modify the `lock_acquire` function and `lock_release` functions. Before a thread acquires a lock, it first saves itself to the donation list. This step ensures that the thread's priority can be donated to the thread holding the lock.

Once the current thread is stored in the donation list, we proceed with the priority donation process. This process ensures that a thread with a lower priority will receive a higher priority from donating threads. However, to avoid excessive depth in the priority donation, we limit the depth to 8, as suggested by the Pintos documentation. Therefore, the priority donation will be propagated a maximum of 8 times. This prevents performance degradation and ensures that the priority donation mechanism remains efficient.

When the lock is released, the priority of the thread holding the lock needs to be restored to its original value. Before the `sema_up` operation (in `lock_release`), we will reverse the donation process. Specifically, the thread that releases the lock will adopt the lowest priority from the donation list. Furthermore, all threads in the donation list will have their original priority restored, ensuring that no thread retains an unnecessary donation after the lock is released (goal #4).

3. Advanced Scheduler

The Multi-Level Feedback Queue Scheduler (MLFQS) is a scheduling mechanism that adjusts the priorities of threads dynamically based on various factors such as CPU usage and niceness. In our implementation, we follow the process outlined in Appendix B of the Pintos documentation.

Initially, the user will select between the priority scheduler or the advanced scheduler. The selection is made using the `thread_mlfqs` Boolean variable, which is set to `True` when the advanced scheduler (MLFQS) is being used.

To implement the MLFQS, we need to calculate the following key values:

1. Niceness (nice): Represents how "nice" a thread is toward other threads, with lower values allowing it to take more CPU time.
2. Priority: This determines the order in which threads are scheduled, with higher priority threads being scheduled first.
3. Recent CPU: Tracks how much CPU time a thread has used recently.
4. Load Average: Represents the system's average load and is used to adjust the recent CPU usage.

These values form the basis of the MLFQS algorithm, and their calculation is critical for maintaining proper thread scheduling. It is important to note that while priority, nice, and `ready_threads` are integer values, both `recent_cpu` and `load_avg` are floating-point values. Thus, we will need to carefully implement the floating-point arithmetic necessary for calculating these values. The formulas for calculating them can be found on page 95 of the Pintos documentation.

We will first introduce four new variables to the thread structure: `nice`, `priority`, `recent_cpu`, and `load_avg`. The `nice` variable represents the niceness of the thread, affecting how willing it is to yield CPU time to other threads. The `priority` determines

the scheduling order of threads, with higher-priority threads being executed first.

The `recent_cpu` tracks the CPU time used by each thread recently, while the `load_avg` represents the system-wide load average, influencing the scheduling decisions for all threads.

Next, we will implement the formulas required to calculate these values based on the guidelines provided in the Pintos documentation. The priority of each thread will be recalculated every four clock ticks, ensuring that it adapts dynamically according to the thread's recent CPU usage and niceness. The `recent_cpu` value will be incremented by one for the running thread on every clock tick, reflecting the continuous CPU usage by active threads. Additionally, both the `load_avg` and `recent_cpu` will be recalculated once per time tick, ensuring that system-wide metrics like the load average are regularly updated to adjust scheduling decisions effectively.

To manage these calculations, we will create separate functions to handle the updates for `nice`, `priority`, `recent_cpu`, and `load_avg`. These functions will be based on the formulas outlined in the Pintos documentation. The recalculation of these values ensures that the Multi-Level Feedback Queue Scheduler (MLFQS) operates efficiently and in accordance with real-time system activity.

First, we will implement the `thread_set_nice` function, which addresses the following formula.

$$\text{priority} = \text{PRI_MAX} - (\text{recent_cpu} / 4) - (\text{nice} * 2)$$

Secondly, `recent_cpu` is calculated in `thread_get_recent_cpu` as such:

$$\text{recent_cpu} = (2 * \text{load_avg}) / (2 * \text{load_avg} + 1) * \text{recent_cpu} + \text{nice},$$

Lastly, load avg is calculated in thread_get_load_avg as such:

$$\text{load_avg} = (59/60) * \text{load_avg} + (1/60) * \text{ready_threads}$$

Most importantly, the thread_set_priority function should not be executed in this case. Therefore, if MLFQS (Multi-Level Feedback Queue Scheduler) is active, the thread_set_priority function should simply return without making any changes.

Additionally, recent_cpu and load_avg are real numbers; however, Pintos does not support floating-point arithmetic in the kernel. As a result, we need to implement the calculations using fixed-point arithmetic.

Convert n to fixed point:	$n * f$
Convert x to integer (rounding toward zero):	x / f
Convert x to integer (rounding to nearest):	$(x + f / 2) / f$ if $x \geq 0$, $(x - f / 2) / f$ if $x < 0$.
Add x and y :	$x + y$
Subtract y from x :	$x - y$
Add x and n :	$x + n * f$
Subtract n from x :	$x - n * f$
Multiply x by y :	$((\text{int64_t}) x) * y / f$
Multiply x by n :	$x * n$
Divide x by y :	$((\text{int64_t}) x) * f / y$
Divide x by n :	x / n

Finally, we will modify the timer_interrupt function in timer.c to incorporate these calculations for priority. An if-else statement will be used to check whether the current scheduling mode is MLFQS or the priority scheduler. In MLFQS mode, the necessary calculations for recent_cpu, priority, and load_avg will be triggered at each clock tick or time tick as appropriate.